

Wireshark Network Traffic Analysis

Investigating Suspicious Network Activity Through Packet Analysis

SOC / Cybersecurity Homelab Project

Environment	Oracle VirtualBox isolated lab
Systems	DC01 - 192.168.100.10 WS01 - 192.168.100.20 KALI01 - 192.168.100.30
Primary Tool	Wireshark
Supporting Tools	Kali Linux, Nmap, Windows 10, Windows Server 2025, Splunk Universal Forwarder
Capture	Project5_Network_Traffic.pcapng - 1,089 packets

1. Project Objective

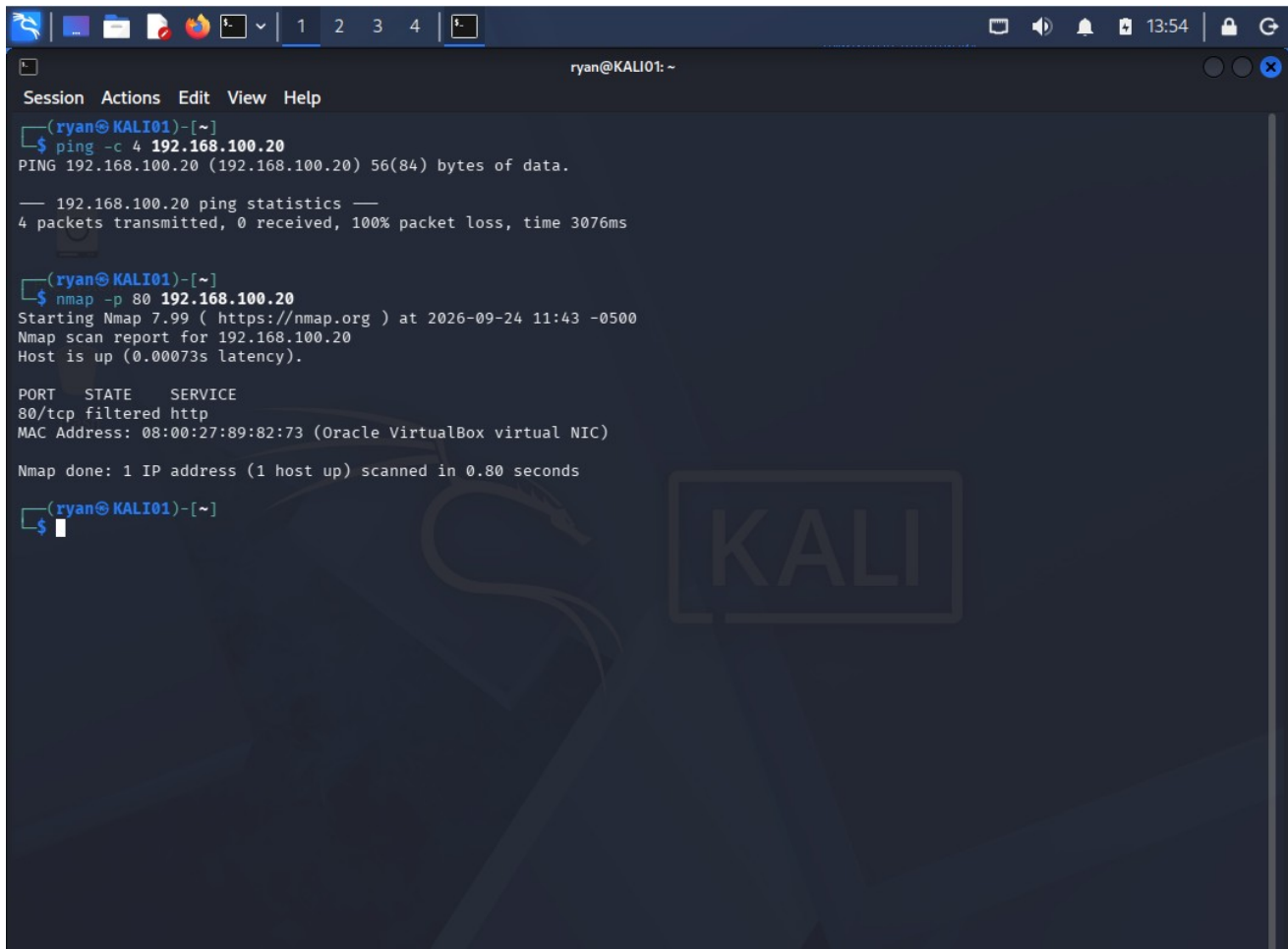
The goal of this project was to gain hands-on experience with Wireshark and connect previously studied networking concepts to real packet data. Network activity was generated from KALI01 toward WS01 using ICMP Echo Requests and a TCP port 80 probe. The resulting traffic was captured on WS01 and investigated with Wireshark display filters, protocol statistics, conversation analysis, TCP stream reconstruction, DNS analysis, and Expert Information.

The investigation also examined normal background traffic between WS01 and DC01. This provided a comparison between unanswered probing activity and successful, expected communications such as Splunk forwarding and DNS resolution.

2. Lab Architecture and Traffic Generation

- DC01 - Windows Server 2025 domain controller and Splunk Enterprise receiver - 192.168.100.10
- WS01 - Windows 10 domain workstation and Wireshark capture point - 192.168.100.20
- KALI01 - Kali Linux testing system - 192.168.100.30
- HOME / home.local Active Directory lab on the 192.168.100.0/24 internal network

KALI01 generated four ICMP Echo Requests and a basic Nmap probe against TCP port 80 on WS01. Kali reported 100% packet loss for the ping and Nmap classified port 80 as filtered. The packet capture later demonstrated that the requests reached WS01 even though no corresponding responses were observed.

A screenshot of a Kali Linux terminal window. The window title is 'ryan@KALI01: ~'. The terminal shows the following commands and output:
1. Command: `ping -c 4 192.168.100.20`
Output: `PING 192.168.100.20 (192.168.100.20) 56(84) bytes of data.`
`— 192.168.100.20 ping statistics —`
`4 packets transmitted, 0 received, 100% packet loss, time 3076ms`
2. Command: `nmap -p 80 192.168.100.20`
Output: `Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-24 11:43 -0500`
`Nmap scan report for 192.168.100.20`
`Host is up (0.00073s latency).`

PORT	STATE	SERVICE
80/tcp	filtered	http

`MAC Address: 08:00:27:89:82:73 (Oracle VirtualBox virtual NIC)`
`Nmap done: 1 IP address (1 host up) scanned in 0.80 seconds`
3. The prompt `(ryan@KALI01)-[~]` is shown with a cursor, indicating the terminal is ready for the next command.

Figure 1. KALI01 traffic generation: four ICMP Echo Requests and an Nmap TCP/80 probe.

3. Capture Overview

The capture contained 1,089 packets. Protocol Hierarchy Statistics were reviewed for traffic involving WS01. The filtered hierarchy contained 1,081 packets and showed that TCP dominated the capture, with additional UDP, DNS, LDAP, Kerberos, DCE/RPC, Active Directory replication, data, and ICMP traffic. This established a baseline before focusing on individual communications.

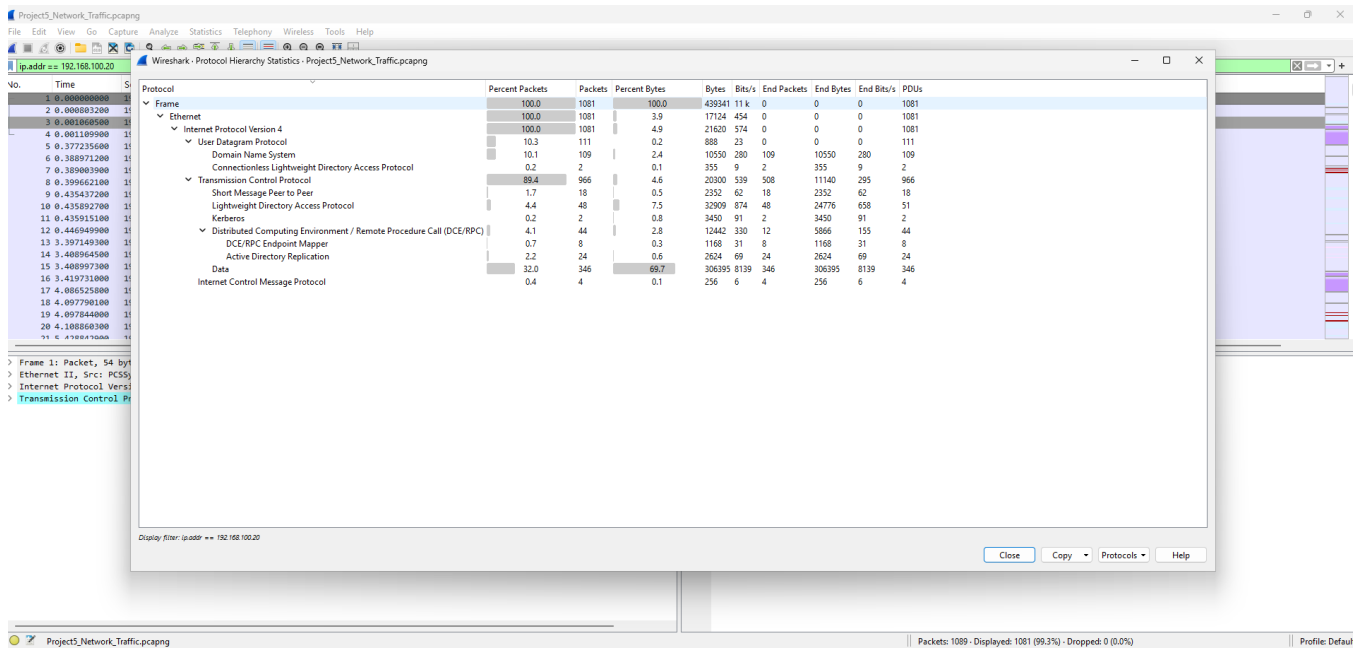


Figure 2. Protocol Hierarchy Statistics for traffic involving WS01 (192.168.100.20).

4. Investigation Findings

4.1 ICMP Echo Requests Without Replies

The display filter `ip.addr == 192.168.100.20 && icmp` isolated four ICMP Echo Requests from KALI01 (192.168.100.30) to WS01 (192.168.100.20). No Echo Replies were present. The capture therefore confirmed that the requests reached WS01's network interface, while avoiding an unsupported conclusion about the exact mechanism that prevented a reply.

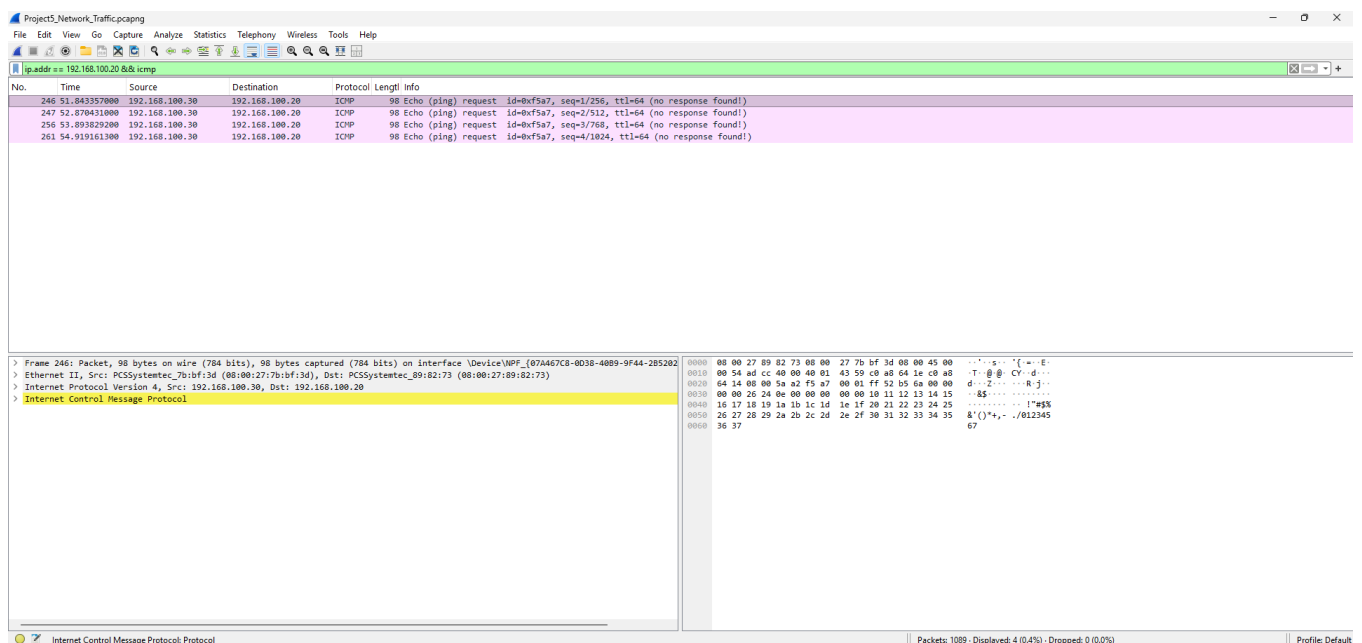


Figure 3. Four ICMP Echo Requests from KALI01 to WS01 with no observed replies.

4.2 TCP Port 80 SYN Probes

Filtering for `ip.addr == 192.168.100.20 && tcp.port == 80` revealed two TCP SYN packets from KALI01 to WS01 destination port 80. The probes used different ephemeral source ports (54976 and 54978), and no SYN/ACK or RST response from WS01 was observed. The evidence is consistent with the traffic being silently filtered or dropped, although the PCAP alone does not identify the specific filtering mechanism.

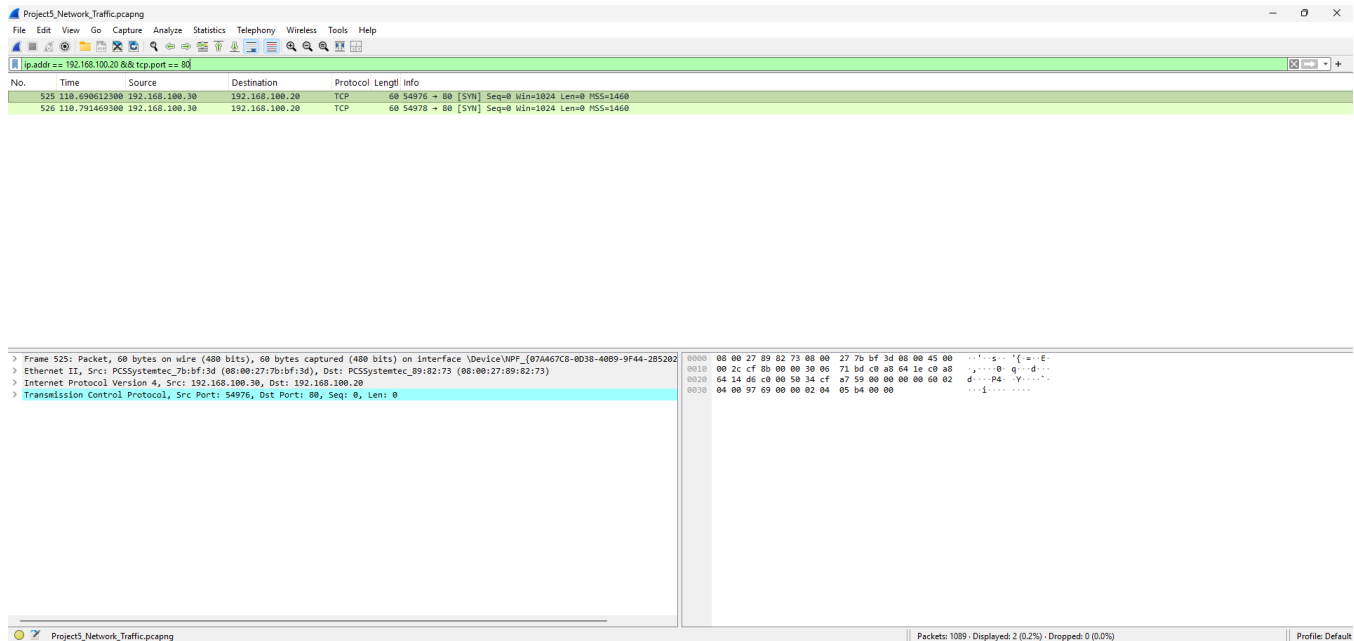


Figure 4. Two TCP SYN probes from KALI01 to WS01 port 80 with no observed response.

4.3 Successful Splunk TCP Communication

Normal traffic between WS01 and DC01 provided a useful contrast. A TCP conversation to DC01 port 9997 contained a complete three-way handshake: SYN, SYN/ACK, and ACK. This confirmed successful TCP session establishment.

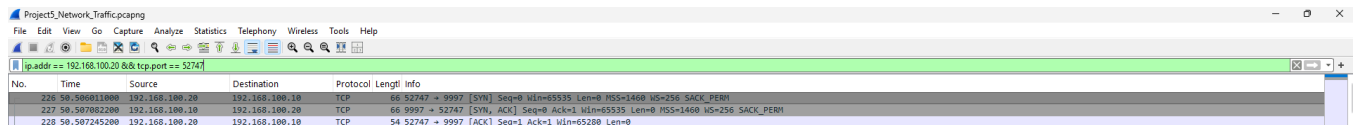


Figure 5. Successful TCP three-way handshake between WS01 and DC01 on Splunk port 9997.

Following the TCP stream provided stronger application-level identification than relying on the port number alone. Readable strings such as `splunk-cooked-mode-v3`, `ForwarderInfo`, and references to the Splunk Universal Forwarder were visible, supporting identification of the communication as Splunk forwarding traffic.

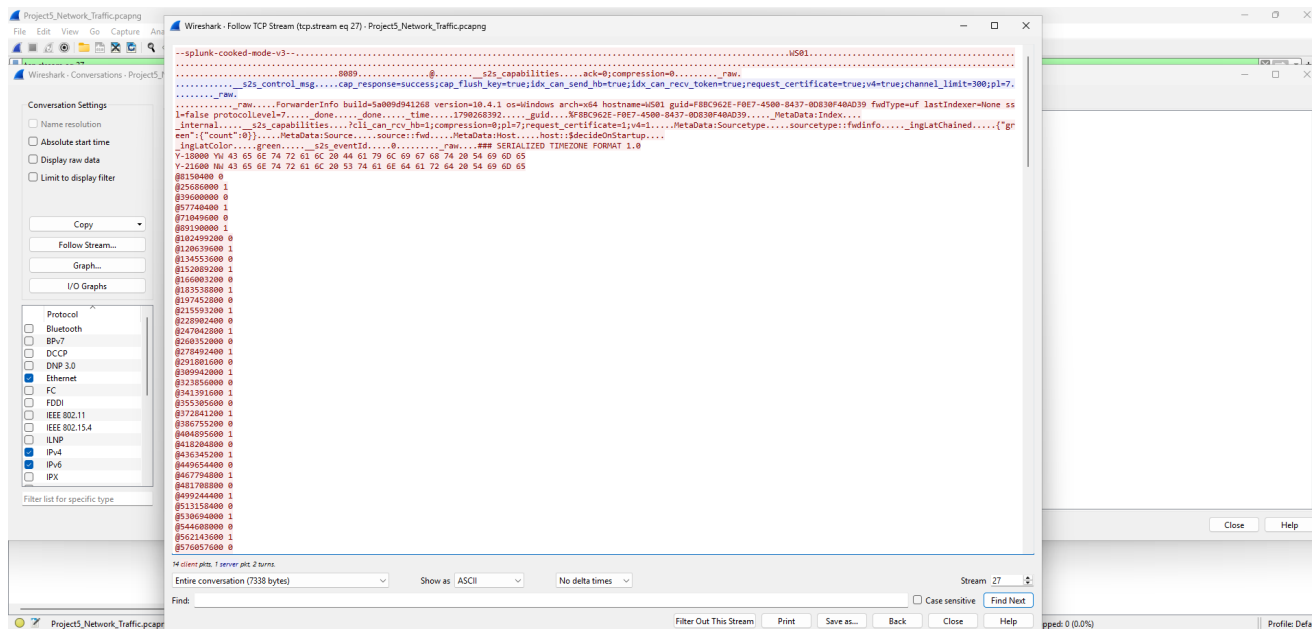


Figure 6. Follow TCP Stream showing Splunk Universal Forwarder application data.

4.4 DNS Query and Response Correlation

DNS analysis showed both routine background resolution and the manually generated lookup for google.com. The query 'A google.com' from WS01 to DC01 used DNS Transaction ID 0x0004. Filtering with 'dns.id == 0x0004' isolated the request and its matching response. The response returned six IPv4 A records, demonstrating how transaction IDs can be used to correlate DNS questions with their exact answers.

The investigation also exposed A records for IPv4, AAAA records for IPv6, and PTR / in-addr.arpa reverse lookups. The google.com exchange appeared consistent with normal DNS resolution; the packet evidence did not indicate suspicious behavior in that transaction.

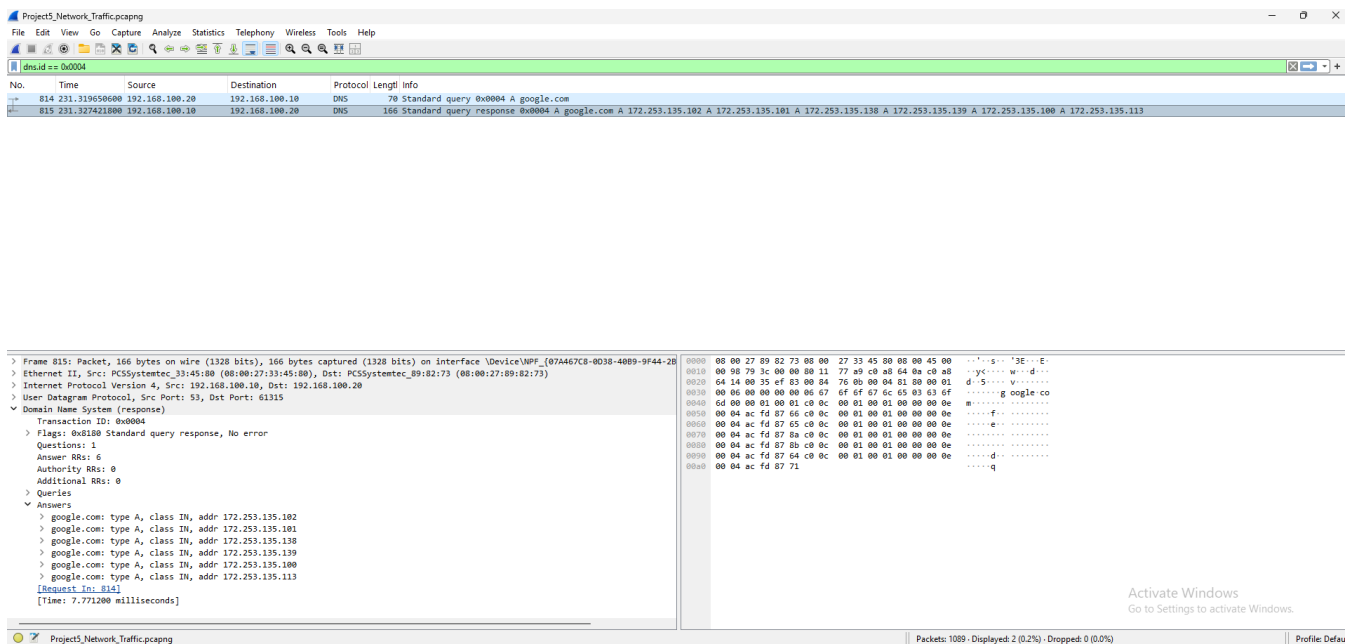
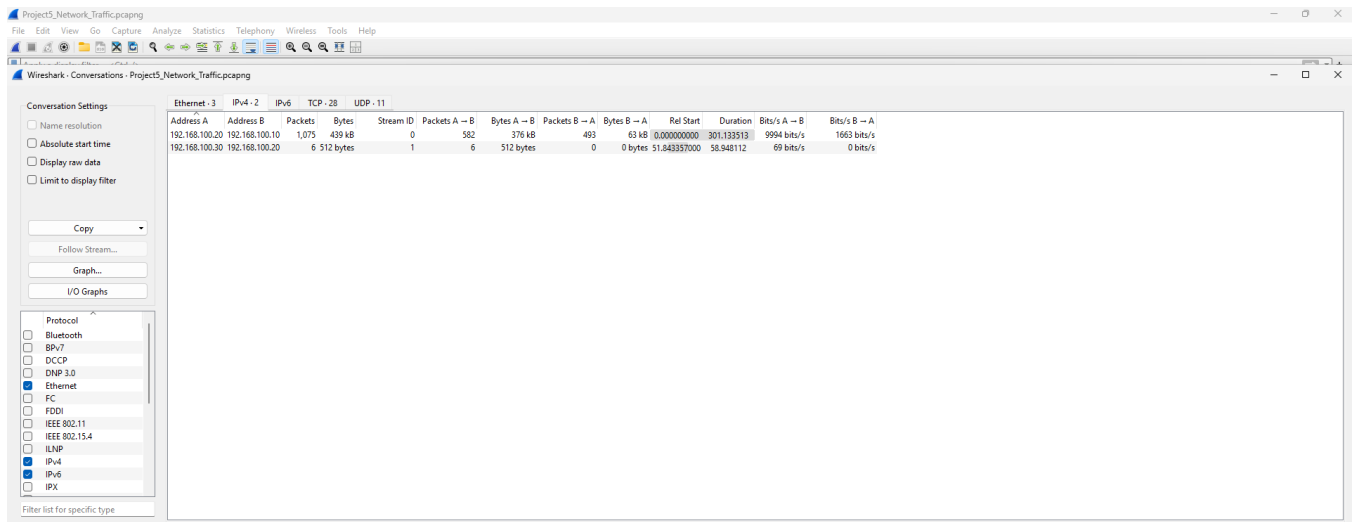


Figure 7. DNS Transaction ID 0x0004 correlating the google.com A query with a six-address response.

4.5 Conversation-Based Host Discovery

Rather than beginning with a known attacker IP, IPv4 Conversation Statistics were used to identify communication patterns that stood out. The dominant WS01-DC01 conversation contained 1,075 packets and approximately 439 kB, with traffic in both directions. In contrast, the KALI01-to-WS01 conversation contained only six packets (512 bytes), all traveling toward WS01 and none returning.

Pivoting from the conversation summary to ``ip.addr == 192.168.100.30`` revealed that the six packets consisted of four ICMP Echo Requests and two TCP SYN probes. This demonstrated an analyst-oriented workflow: use high-level statistics to identify noteworthy communication, then inspect the underlying packets to determine what occurred. The one-way pattern was not treated as malicious by default, but it provided sufficient reason for further analysis.



The screenshot shows the Wireshark 'Conversations' window for 'Project5_Network_Traffic.pcapng'. The 'IPv4' tab is selected, showing a list of conversations. The first conversation is between 192.168.100.20 and 192.168.100.30, with 6 packets and 512 bytes. The 'Conversations' table is as follows:

Address A	Address B	Packets	Bytes	Stream ID	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Duration	Bits/s A → B	Bits/s B → A
192.168.100.20	192.168.100.10	1,075	439 kB	0	582	376 kB	493	63 kB	0.000000000	30.133513	9994 bits/s	1663 bits/s
192.168.100.30	192.168.100.20	6	512 bytes	1	6	512 bytes	0	0 bytes	51.043337000	58.948112	69 bits/s	0 bits/s

On the left, the 'Conversation Settings' panel shows 'Name resolution' and 'Display raw data' checked. The 'Filter list for specific type' section shows 'Ethernet', 'IPv4', and 'IPv6' selected.

Figure 8. IPv4 Conversations highlighting the six-packet, one-way KALI01-to-WS01 relationship.

5. Expert Information and Contextual Analysis

Wireshark Expert Information was used as an additional triage source. No Error-severity findings were present. Warning-level observations included ICMP 'Response not found', a DNS query retransmission, and TCP connection resets. These warnings were treated as investigative leads rather than automatic indicators of malicious activity.

A TCP reset example on the WS01-to-DC01 TCP/389 conversation was examined in context. The closing sequence showed WS01 sending FIN/ACK, DC01 acknowledging the FIN, and DC01 subsequently sending RST/ACK. Reviewing the surrounding packets demonstrated why individual Expert Information warnings should not be interpreted in isolation.

6. Troubleshooting and Lessons Learned

The largest troubleshooting moment occurred during initial traffic testing. Kali reported 100% packet loss for the four pings to WS01, which initially made it appear that the traffic might not be reaching the workstation. Wireshark showed all four Echo Requests arriving at WS01 with no corresponding Echo Replies. This reinforced an important troubleshooting principle: a failed command or connection attempt

does not necessarily mean that packets never reached the destination. Packet capture can reveal where communication succeeded and where the expected response stopped.

Display filtering became straightforward after hands-on exposure. The project also reinforced that Wireshark evaluates display-filter conditions against individual packets, that statistics can change when 'Limit to display filter' is enabled, and that protocol or application identity should be supported by packet evidence rather than inferred from a port number alone.

7. Technical Skills Developed

- Wireshark packet capture and packet-level network analysis
- Display filtering by IP address, protocol, port, TCP flags, and DNS fields
- TCP behavior analysis, including SYN, SYN/ACK, ACK, FIN, and RST
- Identification and validation of a TCP three-way handshake
- Endpoint and Conversation Statistics for host discovery and triage
- Follow TCP Stream for application-level communication analysis
- DNS query/response correlation using Transaction IDs
- Interpretation of A, AAAA, and PTR DNS records
- Use of Expert Information as an investigative lead rather than a verdict
- Evidence-based differentiation between normal-looking traffic and activity warranting further investigation

8. Analyst Takeaways

Before this project, many of the underlying concepts had been encountered through online coursework and certification study. This project provided direct exposure to the packet-level behavior behind those concepts. Seeing a TCP three-way handshake occur between two lab systems, correlating a DNS query with its response, and following an application stream made the networking concepts more concrete.

If presented with an unfamiliar PCAP in an analyst role, an effective starting point would be high-level statistics such as Conversations and Endpoints to understand which systems communicated and identify patterns that warrant further review. From there, display filters can isolate relevant hosts and protocols, TCP behavior can be examined, DNS requests can be correlated with responses, streams can be followed when appropriate, and Expert Information can provide additional leads. The central lesson is to let the packet evidence establish what happened rather than assuming that unusual traffic is automatically malicious.

9. Conclusion

This project established a practical foundation in Wireshark and packet analysis. Controlled traffic from KALI01 was successfully captured and isolated, unanswered ICMP and TCP probes were identified, successful Splunk and DNS communications were validated, and conversation statistics were used to discover noteworthy traffic without beginning from a known source IP. The project bridged theoretical networking knowledge with hands-on investigation and reinforced an evidence-driven approach that is directly relevant to entry-level SOC and cybersecurity analyst work.